A classe `Throwable` e a classe `Exception` formam a base do sistema de tratamento de erros e exceções no Java. Elas definem como problemas, falhas e eventos inesperados são representados, lançados e capturados durante a execução de um programa. [1]

---

1. A Classe `Throwable` (A Raiz de Tudo)

- **O que é:** `java.lang.Throwable` é a superclasse (classe pai) de **todas** as classes de erro e exceção na linguagem Java. [1, 2]
- **O que ela faz:** Concentra os métodos e atributos fundamentais para registrar o estado de um problema, como:
  - `getMessage()`: Retorna a descrição textual do erro.
  - `printStackTrace()`: Imprime no console a pilha de execução (*stack trace*), mostrando exatamente em quais classes e linhas o problema ocorreu.
  - `getCause()`: Retorna a causa raiz (outro `Throwable` que originou este). [1]
- **Quem herda dela:** A `Throwable` possui duas ramificações principais:
  A. **`Error`**: Representa problemas graves na Máquina Virtual Java (JVM), como falta de memória (`OutOfMemoryError`) ou estouro de pilha (`StackOverflowError`). A aplicação **não** deve tentar capturar ou tratar esses erros.
  B. **`Exception`**: Representa condições anormais que a aplicação **pode** (e muitas vezes deve) tentar capturar e tratar. [1, 2]

---

2. A Classe `Exception` (O Foco do Tratamento)

- **O que é:** `java.lang.Exception` é a subclasse direta de `Throwable` e serve como raiz para todas as exceções recuperáveis no código.
- **O que ela faz:** Diferencia o que é um "erro catastrófico da JVM" (`Error`) de um "problema da aplicação" (`Exception`). Ela se divide em dois grandes grupos: [1, 2, 3]

Checked Exceptions (Exceções Verificadas)

- **O que são:** Filhas diretas de `Exception` (com exceção da `RuntimeException`).
- **Como funcionam:** O compilador Java **obriga** você a tratar ou declarar esses erros. Se você chamar um método que lança uma `IOException` ou `SQLException`, o código não compila a menos que você o envolva em um bloco `try-catch` ou use a palavra-chave `throws` na assinatura do método.
- **Exemplo:** `FileNotFoundException` (tentar abrir um arquivo que não existe no disco). [1, 2, 3]

Unchecked Exceptions (`RuntimeException`)

- **O que são:** Subclasses de `RuntimeException` (que por sua vez herda de `Exception`).
- **Como funcionam:** O compilador **não** exige que você trate explicitamente essas exceções. Elas geralmente representam **bugs de lógica** do programador.
- **Exemplos:** `NullPointerException` (tentar usar um objeto `null`), `ArrayIndexOutOfBoundsException` (acessar uma posição inválida de um array), ou `ArithmeticException` (como divisão por zero). [1, 2, 3, 4, 5]

---

3. Como funciona na prática (`try`, `catch`, `throw`, `throws`)

- **`throw`**: Usado dentro do método para **lançar** manualmente uma instância de um `Throwable` (ou suas subclasses), por exemplo: `throw new IllegalArgumentException("Valor inválido");`.
- **`throws`**: Usado na assinatura do método para **avisar** quem o chamou que ele pode disparar uma exceção do tipo `Exception` (ou específica).
- **`try-catch`**: Utilizado para **capturar** e conter a exceção, evitando que o programa feche abruptamente e permitindo executar um plano B. [1, 2, 3]

A classe **`Error`** é a outra grande ramificação que herda diretamente de `Throwable` (ao lado de `Exception`).

Enquanto as exceções representam condições que uma aplicação normal pode e deve tentar tratar, a classe **`Error`** **representa falhas graves, críticas e catastróficas a nível de infraestrutura ou da própria Máquina Virtual Java (JVM)**.

---

1. Filosofia do `Error`

A regra de ouro em Java é: **Você não deve tentar capturar (`catch`) ou tratar um `Error`.**

Quando um `Error` acontece, significa que a JVM ou o ambiente de execução entrou em um estado tão instável que a aplicação geralmente não tem recursos ou condições saudáveis para se recuperar. A atitude correta quase sempre é deixar o programa encerrar para que o problema seja corrigido na infraestrutura ou no código.

---

2. Principais Características

- **São Não-Verificados (*Unchecked*):** Assim como a `RuntimeException`, o compilador do Java não obriga você a declarar um `Error` com a palavra-chave `throws` e nem a colocá-lo dentro de um bloco `try-catch`.
- **Origem Externa à Lógica Comum:** Geralmente não são causados por um erro simples de digitação ou validação de dados (como um ID inválido), mas sim por

esgotamento de recursos do sistema ou incompatibilidade de arquivos compilados.

---

3. Exemplos Comuns de `Error`

- **`OutOfMemoryError`:** A JVM ficou sem memória RAM disponível para alocar novos objetos e o coletor de lixo (*Garbage Collector*) não conseguiu liberar mais espaço.
- **`StackOverflowError`:** A pilha de execução do Java estourou. O motivo mais comum é uma função recursiva que entrou em loop infinito e nunca atinge a condição de parada.
- **`NoClassDefFoundError`:** A JVM tentou carregar a definição de uma classe que existia durante a compilação, mas que desapareceu ou não foi encontrada no *classpath* em tempo de execução.
- **`InternalError`:** Um erro inesperado e incomum dentro da própria implementação da Máquina Virtual Java.

---

Resumo Comparativo: `Exception` vs `Error`

| **Critério** | **`Exception`** | **`Error`** |
|---|---|---|
| **Origem** | Erros na lógica do código ou condições externas previsíveis (ex: rede fora do ar, arquivo faltando). | Falhas graves na J recursos do siste |
| **Recuperável?** | **Sim.** A aplicação consegue tratar e continuar rodando. | **Não.** A aplicação ambiente se torn |
| **Exemplos** | `NullPointerException`, `IOException`, `SQLException`. | `OutOfMemoryErr` |
| **Boa prática** | Capturar com `try-catch` ou lançar adiante com `throws`. | Deixar o program configurações do |

A classe `Exception` possui **centenas de subclasses** nativas no Java, divididas principalmente entre exceções que você é obrigado a tratar (**Checked**) e aquelas ligadas a erros de lógica (**Unchecked** / `RuntimeException`).

As subclasses mais importantes e utilizadas no dia a dia do desenvolvimento Java estão organizadas abaixo por categoria:

---

1. Subclasses de `RuntimeException` (Exceções Não-Verificadas)

Representam **erros de lógica ou bugs** cometidos pelo programador. O compilador não obriga o tratamento, pois o correto é corrigir o código.

- **`NullPointerException` (NPE):** Disparada quando o código tenta acessar um método ou atributo de uma referência que está valendo `null`.
- **`IllegalArgumentException`:** Lançada quando um método recebe um argumento com formato ou valor inválido (ex: passar uma idade negativa).
- **`ArrayIndexOutOfBoundsException`:** Ocorre ao tentar acessar um índice de um array que não existe (ex: acessar a posição 5 de um array que só vai até 3).
- **`ArithmeticException`:** Erros em operações matemáticas, sendo o caso mais famoso a divisão por zero.
- **`ClassCastException`:** Ocorre ao tentar converter (*cast*) um objeto para um tipo de classe do qual ele não é herdeiro.
- **`NumberFormatException`:** Subclasse de *IllegalArgumentException*, disparada quando você tenta converter uma String em número, mas o texto contém letras (ex: `Integer.parseInt("123a")`).

---

2. Subclasses de Exceções Verificadas (Checked Exceptions)

Representam falhas que acontecem por **fatores externos ao controle do programa** (como problemas de rede, arquivos ou banco de dados). O Java obriga você a tratá-las com `try-catch` ou declará-las com `throws`.

- **`IOException`:** A mãe de todas as exceções de entrada e saída de dados. É disparada quando ocorre uma falha ao ler ou escrever informações.
  - **`FileNotFoundException`:** Subclasse da `IOException`, ocorre especificamente quando o código tenta abrir um arquivo em um caminho que não existe.
- **`SQLException`:** Agrupa todos os erros relacionados ao acesso e manipulação de bancos de dados (erros de sintaxe SQL, falha de conexão, etc.).
- **`ClassNotFoundException`:** Disparada quando o programa tenta carregar uma classe dinamicamente (usando strings com o nome da classe) mas ela não é encontrada no sistema.
- **`InterruptedException`:** Ocorre quando uma Thread (linha de execução) que estava aguardando ou dormindo (`Thread.sleep()`) é interrompida por outro processo.

---

Representação Visual da Árvore de Exceções

text

```
Throwable
├── Error (Graves: OutOfMemoryError, StackOverflowError...)
└── Exception (Geral)
```

```
        ├── IOException (Checked: Falha de arquivos/rede)
        ├── SQLException (Checked: Falha de banco de dados)
        └── RuntimeException (Unchecked)
            ├── NullPointerException (Bug: objeto nulo)
            ├── ArithmeticException (Bug: divisão por zero)
            └── IllegalArgumentException (Bug: argumento inválido)
```

Para criar uma exceção personalizada em Java, você só precisa criar uma classe comum e fazê-la herdar de alguma classe da família `Throwable`.

A boa prática dita que:

- Se você quer uma exceção **Checked** (verificada/obrigatória), herde de **`Exception`**.
- Se você quer uma exceção **Unchecked** (não-verificada/de lógica), herde de **`RuntimeException`** *(esta é a opção mais usada no desenvolvimento moderno, como no Spring Boot)*.

Abaixo está o exemplo prático de uma exceção do tipo **Unchecked** para um sistema de e-commerce, simulando um erro de saldo insuficiente.

---

1. Criando a Exceção Personalizada

**O padrão do mercado** é criar três construtores para que sua exceção seja flexível (aceitando apenas uma mensagem, aceitando uma mensagem + a causa raiz do erro, ou vazia).

java

```java
// Herdando de RuntimeException para torná-la Unchecked
public class SaldoInsuficienteException extends RuntimeException {

    // Construtor padrão sem mensagem
    public SaldoInsuficienteException() {
        super();
    }

    // Construtor que aceita uma mensagem customizada (Mais utilizado)
    public SaldoInsuficienteException(String mensagem) {
        super(mensagem);
    }

    // Construtor que aceita a mensagem e a causa original do erro (Chaining Exceptions)
    public SaldoInsuficienteException(String mensagem, Throwable causa) {
        super(mensagem, causa);
    }
```

Use o código com cuidado.

---

2. Utilizando a Exceção na Lógica do Negócio

Aqui usamos a palavra-chave **`throw`** para disparar a exceção manualmente caso a regra de negócio seja violada.

java

```
public class ContaBancaria {
    private double saldo = 100.0;

    public void sacar(double valor) {
        if (valor > this.saldo) {
            // Disparando a nossa exceção personalizada se o saldo for menor
que o saque
            throw new SaldoInsuficienteException("Falha no saque: Seu saldo
atual é R$" + this.saldo + ", mas você tentou sacar R$" + valor);
        }

        this.saldo -= valor;
        System.out.println("Saque realizado com sucesso! Saldo restante: R$" +
this.saldo);
    }
}
```

Use o código com cuidado.

---

3. Capturando a Exceção (`try-catch`)

Por fim, a classe principal que executa o código e captura o erro utilizando o bloco **`try-catch`**.

java

```
public class Sistema {
    public static void main(String[] args) {
        ContaBancaria conta = new ContaBancaria();

        try {
            // Tentativa de saque válido
            conta.sacar(30.0);

            // Tentativa de saque INVÁLIDO (vai disparar o erro)
            conta.sacar(150.0);

            System.out.println("Esta linha não será executada porque o erro
```

```
        acontece antes.");

                } catch (SaldoInsuficienteException e) {
                    // Capturando especificamente a nossa exceção criada
                    System.err.println("Aviso do Sistema: " + e.getMessage());
```

Use o código com cuidado.

Saída no Console ao executar o código:

text

```
Saque realizado com sucesso! Saldo restante: R$70.0
Aviso do Sistema: Falha no saque: Seu saldo atual é R$70.0, mas você tentou sacar R$150.0
```

Use o código com cuidado.